Check Org User has Feature Accessibility - From Org Centralize service (Key-Cloak)

Audience:

> This document is intended for internal WhiteHelmet teams:
>
> - **Squad/backend engineers** implementing services that need to check whether a feature or service is enabled for a given user under a specific organisation.
> - **Platform / identity engineers** configuring Keycloak realms, groups, and attributes so that feature flags resolve correctly.
> - **SREs / platform operators** who need to understand expected request/response patterns and errors for monitoring, logging, and troubleshooting.
>
> Assumed knowledge:
>
> - Basic understanding of HTTP APIs (methods, headers, query parameters, JSON)
> - Familiarity with Keycloak concepts (realms, groups, attributes, bearer tokens)
> - Basic OAuth2/OIDC concepts and how to obtain an access token from Keycloak

Scope:

> This document specifies the internal HTTP “feature-flag check” API exposed by Keycloak as a custom realm resource for WhiteHelmet squads. It covers:
>
> - The endpoint URL and HTTP method
> - Required query parameters and their semantics
> - Authentication requirements
> - Response formats for success and error cases
> - High-level processing rules and expectations on Keycloak group attributes
>
> This API is used by **internal services** to check whether a given feature/service is enabled for a specific user under a specific organization (client).

Table Of Content:

Feature-flag check API

HTTP API exposed from **Key-cloak** (typically a custom realm resource). It reports whether a **feature** is **enabled** for a **client** identified by a UUID stored on a **group** attribute. Feature flags live on the same group as attributes whose keys **start with** `enabled_`.

Concept	In Keycloak

`client_id`	UUID For (Org_id) in a **group attribute**; identifies which group (organization) supplies the flags.
`feature`	Resolved to a group attribute key `enabled_<name>` (see **Query parameters**).

Endpoint

Method: `GET`

URL template:

```
https://{keycloak-host}/realms/{realm}/feature-flags/check
```

Part	Description
`{keycloak-host}`	Keycloak base URL (e.g. `auth.example.com`).
`{realm}`	Realm name.

Some deployments prepend a provider-specific segment to the path; clients should use the **base URL** supplied for their environment. Query parameters and JSON response shapes are as documented below.

Query parameters

Parameter	Required	Description
`client_id`	Yes	UUID string. Must equal the **group attribute** value that links the organisation group to this client (attribute name is deployment-specific, e.g. `client_id` or `o`

		`rganization_clien t_id`).
`feature`	Yes	Feature **logical name** without the `enabled_` prefix. Example: `dark_mode` → the API reads group attribute `enabled_da rk_mode`. The underlying Keycloak attribute key must start with `enabled_`.

Example request:

```
GET https://auth.example.com/realms/acme/feature-flags/check?
client_id=3fa85f64-5717-4562-b3fc-2c963f66afa6&feature=dark_mode
```

Authentication

Header	Role
`Authorization: Bearer <access_token>`	**Required** unless the deployment explicitly allows anonymous access. JWT from the same Keycloak realm (or configured trusted issuer). Required scopes or client roles depend on realm policy.

No other authentication headers are part of this contract. Additional mechanisms (API keys, mTLS) are extensions outside this document.

Summary: callers send `Authorization: Bearer <token>`; anonymous access is off unless the operator documents an exception.

Responses

All bodies are **JSON** with `Content-Type: application/json`.

200 OK — feature resolved

The group for `client_id` was found and the `enabled_*` attribute was evaluated. If the attribute is **missing**, `enabled` is `false`.

Enabled:

```
{
  "client_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "feature": "dark_mode",
  "attribute": "enabled_dark_mode",
  "enabled": true
}
```

Disabled:

```
{
  "client_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "feature": "dark_mode",
  "attribute": "enabled_dark_mode",
  "enabled": false
}
```

400 Bad Request — missing or invalid input

```
{
  "error": "invalid_request",
  "error_description": "client_id and feature are required"
}
```

401 Unauthorized — missing or invalid Bearer token

```
{
  "error": "unauthorized",
  "error_description": "Invalid or missing bearer token"
}
```

404 Not Found — unknown client / group

No group has the given `client_id` attribute value.

```
{
  "error": "not_found",
  "error_description": "No group found for client_id"
}
```

422 Unprocessable Entity — invalid feature name

`feature` does not map to an allowed `enabled_*` attribute (for example invalid characters or naming convention violation).

```
{
  "error": "invalid_feature",
```

```
  "error_description": "feature must map to a group attribute key starting with enabled_"
}

```

Request / response cheat sheet

Item	Value
Method	`GET`
Path	`/realms/{realm}/feature-flags/check`
Query	`client_id=<uuid>&feature=<logical_name>`
Auth	`Authorization: Bearer <token>`
Success body	JSON with `enabled` boolean and echo of inputs

Processing rules

- Resolve the **group** whose client-binding attribute equals `client_id`.
- Read attribute `enabled_<feature>` (or the deployment's agreed full key).
- Parse boolean values from Keycloak string attributes (`true` / `false` with agreed case handling).